



CLAUDE CODE · FUNDAMENTOS · AULA 3

Configuração

com /init

Do projeto zero ao ambiente configurado e pronto: um comando que analisa o projeto, gera o CLAUDE.md e monta a pasta `.claude/`.

/init

settings.json

permissões

/commands



O que é o */init*?

Um comando que analisa seu projeto e cria toda a estrutura de configuração para o Claude.



Cria CLAUDE.md

Analisa código e configs e gera um CLAUDE.md pré-preenchido com o contexto real.



Cria .claude/

Pasta oculta com `settings.json`, `commands/` e arquivos de config.



Define permissões

Configura as permissões base pelo tipo de projeto — segurança por padrão.



Detecta o stack

Identifica linguagem, frameworks, gerenciadores e scripts — zero config manual.

Um */init* bem configurado multiplica a produtividade em todas as próximas aulas

Executando o */init*

1

Inicia o Claude Code

`cd meu-projeto && claude` — entre na pasta primeiro

2

Executa */init*

O Claude começa a varredura imediatamente

3

Análise automática

Lê `package.json`, `requirements.txt`, `go.mod`... detecta stack e estrutura

4

CLAUDE.md gerado

Escreve o CLAUDE.md com o que encontrou — revise e complete

✓

Estrutura *.claude/* criada

`settings.json` + `commands/` prontos — projeto configurado

O que o `/init` *cria*

```
meu-projeto/  
├── CLAUDE.md  
├── .claude/  
│   ├── settings.json  
│   ├── commands/  
│   │   └── deploy.md  
│   └── todos.md  
├── src/  
├── tests/  
└── package.json
```

CLAUDE.md

Contexto do projeto — lido a cada início.

settings.json

Permissões (allow/deny), env e comportamento.

commands/

Cada `.md` vira um comando `/nome` na sessão.

commands/deploy.md

Define o `/deploy` — instruções que o Claude executa.

todos.md

Tarefas pendentes detectadas no projeto (opcional).

Controle *total*

Define o que o Claude pode e não pode fazer no projeto.

```
{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git *)",
      "Read(**)",
      "Write(src/**)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Write(.env)"
    ]
  },
  "env": { "NODE_ENV": "development" }
}
```

✅ permissions.allow

O que o Claude PODE fazer — use glob patterns: `Bash(cmd)`, `Read(**)`, `Write(src/**)`.

🚫 permissions.deny

O que NUNCA pode — tem **prioridade sobre allow**. Protege arquivos e comandos destrutivos.

🌱 env

Variáveis para a sessão.

Não use para segredos reais.

Allow & Deny

Bash(cmd)

Executar comandos (* = wildcard)

Read(path)

** = qualquer pasta

Write(path)

Criar/modificar arquivos

WebFetch(url)

Acessar URLs externas

✓ Allow

- ✓ Bash(npm run test)
- ✓ Bash(git add *)
- ✓ Read(**)
- ✓ Write(src/**)

✗ Deny

- ✗ Bash(rm -rf *)
- ✗ Bash(sudo *)
- ✗ Write(.env)
- ✗ Write(*.key)

Comandos *customizados*

Crie seus próprios comandos /nome — cada arquivo .md em .claude/commands/ vira um comando.

1 Crie um .md
em .claude/commands/

2 Nomeie
deploy.md → vira o /deploy

3 Escreva as instruções
em Markdown

4 Use na sessão
/deploy → o Claude executa tudo

```
# /deploy – deploy.md
Execute o deploy completo:
1. Rode os testes:
  npm run test
2. Se passou, faça build:
  npm run build
3. Commite:
  git commit -m "chore: deploy"
4. Push para main:
  git push origin main
```

/init por *tipo de projeto*

Clique nas abas para ver o que o Claude detecta e configura.

Node / React

Python

Go

Monorepo

Detecta

package.json · node_modules · npm/yarn/pnpm

Allow

npm run * · npx * · node *

Comandos

test → npm test · build → npm run build

Detecta

requirements.txt · pyproject.toml · venv/

Allow

python * · pip install * · pytest

Comandos

test → pytest -v · run → python main.py

Detecta

go.mod · go.sum · *.go

Allow

go build * · go test * · go run *

Comandos

test → go test ./... · build → go build

Detecta

múltiplos package.json · workspaces · turbo.json

Allow

npm run * -w * · turbo * · lerna *

Comandos

test → turbo test · build → turbo build

Teste *rápido*

No `settings.json`, você tem `allow: ["Bash(git *)"]` e `deny: ["Bash(git push *)"]`. O Claude pode dar `git push`?

A Sim — está no `allow` via `git *`

B Não — `deny` tem prioridade sobre `allow`

C Só se você aprovar na hora

D Depende do modelo ativo

Resposta: B. A regra `deny` **sempre vence** a `allow`. Mesmo que `Bash(git *)` libere comandos `git` em geral, o `deny: ["Bash(git push *)"]` bloqueia especificamente o `push`. É assim que você libera uma família de comandos e ainda protege os perigosos.



RESUMO DA AULA 3



Projeto *configurado*

1 `cd projeto && claude && /init`
gera CLAUDE.md + .claude/ automaticamente

2 **Revise e complete**
stack exato, regras de negócio, comandos do dia a dia

3 `git add .claude/`
versione a configuração com o projeto

Próxima aula → **Modo Agente e Tarefas Autônomas.**